Árboles Binarios

Introducción a Árboles Binarios

- Definición: Los árboles binarios son estructuras de datos jerárquicas en las que cada nodo tiene, como máximo, dos hijos.
- Estructura: Cada nodo tiene un valor y dos punteros: uno al hijo izquierdo y otro al hijo derecho.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Ventajas de los Árboles Binarios

- Eficiencia en Búsqueda: Los árboles binarios permiten búsquedas eficientes.
- Ordenamiento: Se utilizan para ordenar datos automáticamente.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Estructura de un Nodo en el Árbol Binario

- Nodo en el Árbol Binario: Cada nodo contiene tres partes principales.
- Valor: Almacena el valor del nodo.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Función de Inserción

- Búsqueda de Nodo: Buscar un nodo en el árbol binario.
- Lógica: Lógica de búsqueda recursiva.
- Código de Ejemplo: Código de Búsqueda.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Función de Inserción

```
NodoArbol* insertarRecursivo(NodoArbol* actual, int valor) {
    if (actual == NULL) {
        return crearNodo(valor);
    }

    if (valor < actual->valor) {
        actual->izquierdo = insertarRecursivo(actual->izquierdo, valor);
    } else if (valor > actual->valor) {
        actual->derecho = insertarRecursivo(actual->derecho, valor);
    }

    return actual;
}
```

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Función de Eliminación

- Eliminación de Nodo: Eliminar un nodo del árbol binario.
- Lógica: Lógica de eliminación para diferentes casos.
- Código de Ejemplo: Código de Eliminación.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Función de Eliminación

```
NodoArbol* eliminarRecursivo(NodoArbol* raiz, int valor) {
    if (raiz == NULL) {
        return raiz;
    }

    if (valor < raiz->valor) {
        raiz->izquierdo = eliminarRecursivo(raiz->izquierdo, valor);
    } else if (valor > raiz->valor) {
        raiz->derecho = eliminarRecursivo(raiz->derecho, valor);
    } else {
        // Lógica de eliminación
    }

    return raiz;
}
```

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Recorrido en Orden

- Recorrido en Orden: Recorrer el árbol en orden.
- Código de Ejemplo: Código de Recorrido en Orden.

```
void recorridoEnOrden(NodoArbol* raiz) {
    if (raiz != NULL) {
        recorridoEnOrden(raiz->izquierdo);
        printf("%d ", raiz->valor);
        recorridoEnOrden(raiz->derecho);
    }
}
```

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Recorrido en Preorden y Postorden

```
void recorridoEnPreorden(NodoArbol* raiz) {
    if (raiz != NULL) {
        printf("%d ", raiz->valor);
        recorridoEnPreorden(raiz->izquierdo);
        recorridoEnPreorden(raiz->derecho);
    }
}

void recorridoEnPostorden(NodoArbol* raiz) {
    if (raiz != NULL) {
        recorridoEnPostorden(raiz->izquierdo);
        recorridoEnPostorden(raiz->derecho);
        printf("%d ", raiz->valor);
    }
}
```

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Árbol de Búsqueda (BST)

- Un árbol binario de búsqueda (BST) es una estructura de datos eficiente para realizar búsquedas, inserciones y eliminaciones.

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl

Árbol de Búsqueda (BST)

```
struct NodoBST* buscar(struct NodoBST* raiz, int valor) {
    if (raiz == NULL || raiz->valor == valor) {
        return raiz;
    }

    if (valor < raiz->valor) {
        return buscar(raiz->izquierdo, valor);
    } else {
        return buscar(raiz->derecho, valor);
    }
}
```

Dr. Pablo Ormeño-Arriagada
pablo.ormeno@uvm.cl